

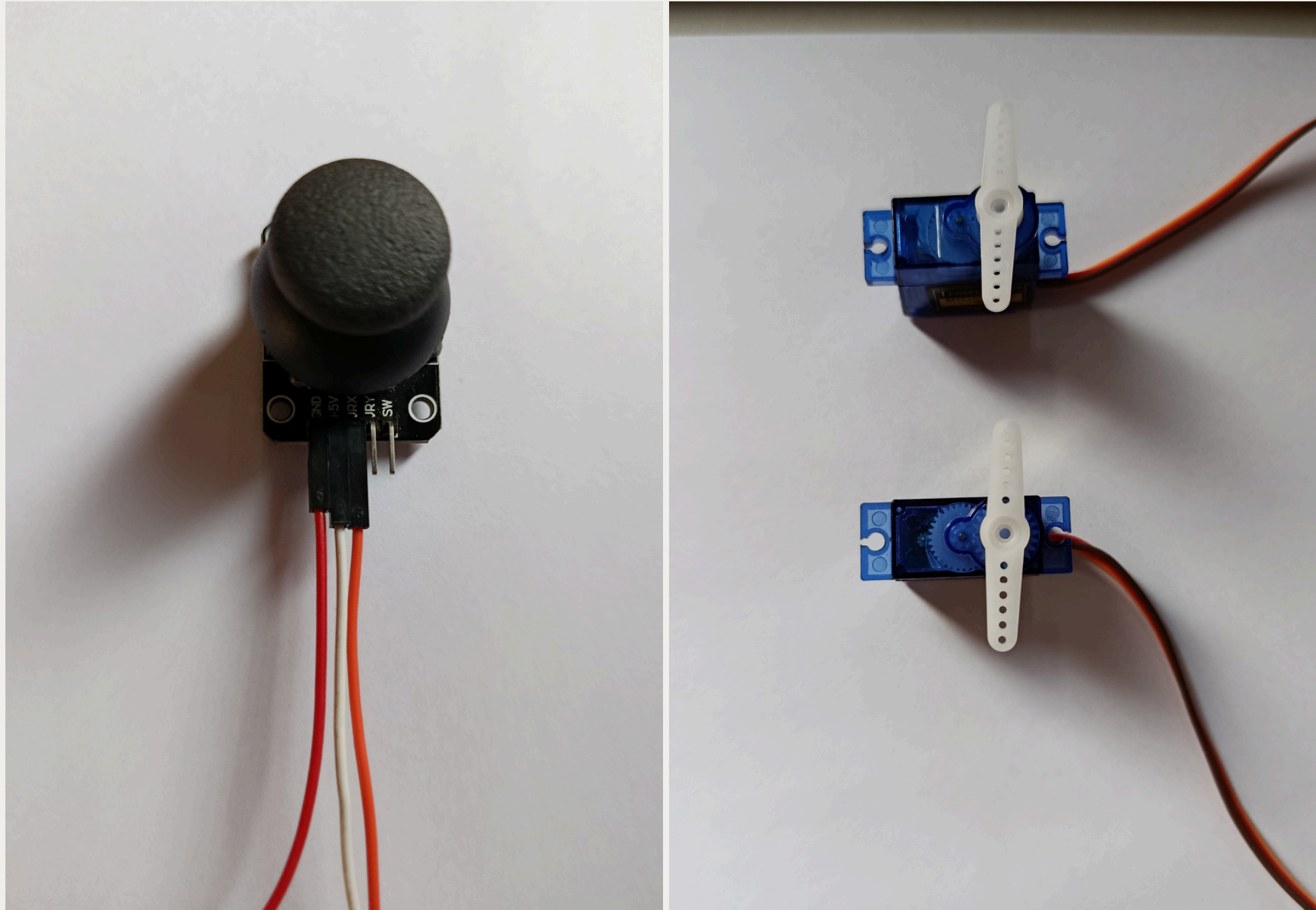
PROYECTO DE ARDUINO

Proyecto:
Servomotores

Presentado por:
Israel Reyes



01 / DEFINIR



¿QUÉ PROBLEMA RESUELVE EL PROYECTO?

- controlar de forma manual, simultánea y coordinada la posición de dos actuadores mecánicos (servomotores) en sentido opuesto usando un único mando físico (joystick).

¿QUÉ DETECTA?

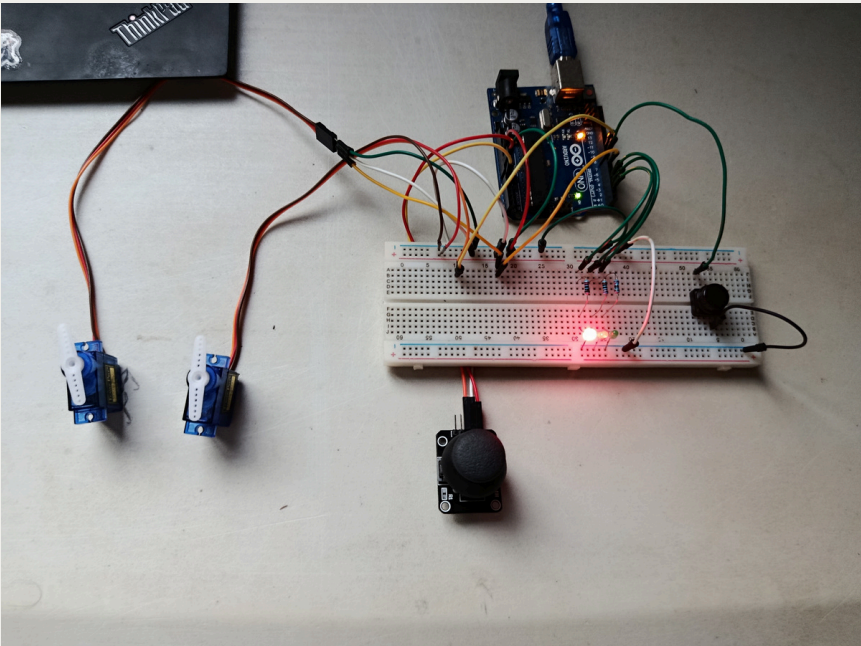
- Mando Analógico (VRX): Sensor potenciométrico que envía lecturas de voltaje variables en el pin A0 (rango de 0 a 1023).
- Pulsador de Seguridad: Entrada digital en el pin 8 con lógica invertida (INPUT_PULLUP).

¿QUÉ HACE?

- Servomotores SG90: Dos actuadores de precisión posicionados en los pines 7 y 9.
- Semáforo de Estado: Tres LEDs indicadores de operación (Pines 2, 3 y 4).



02 / PLANIFICAR



Componente	Cantidad	Tipo de Entrada / Salida	Pin en Arduino
Servomotores	2	Salida (PWM)	Pin 7 y Pin 9
Joystick	1	Entrada Analógica	Pin A0
Botón Pulsador	1	Entrada Digital	Pin 8
LED Rojo	1	Salida Digital	Pin 2
LED Amarillo	1	Salida Digital	Pin 3
LED Verde	1	Salida Digital	Pin 4
Resistencias (220)	3	Protección / Pasivo	(A la Protoboard)



03 / DIAGRAMA

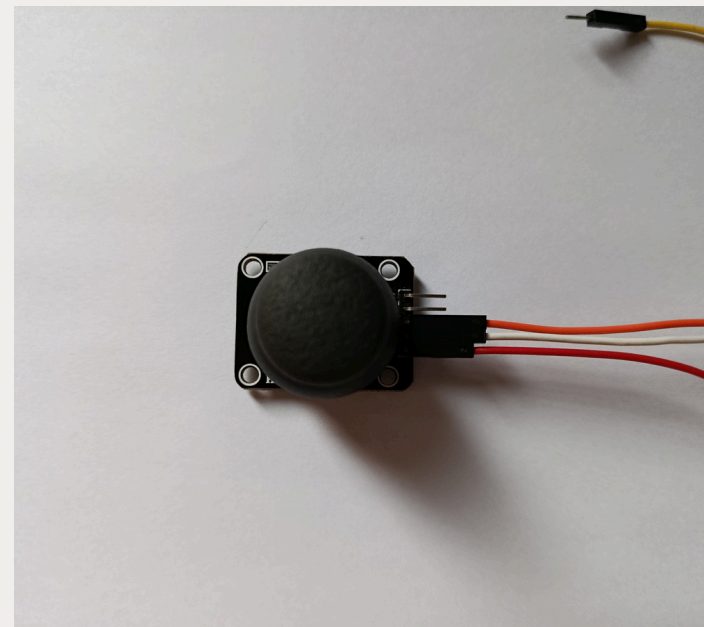
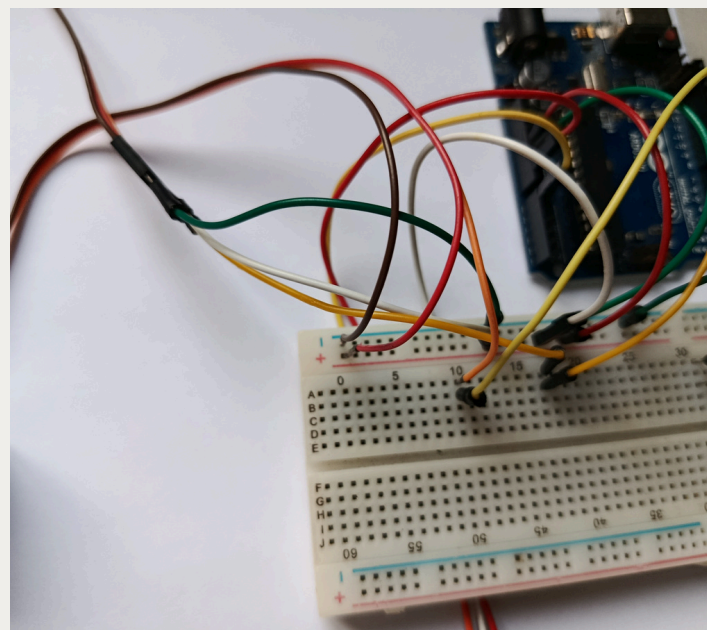
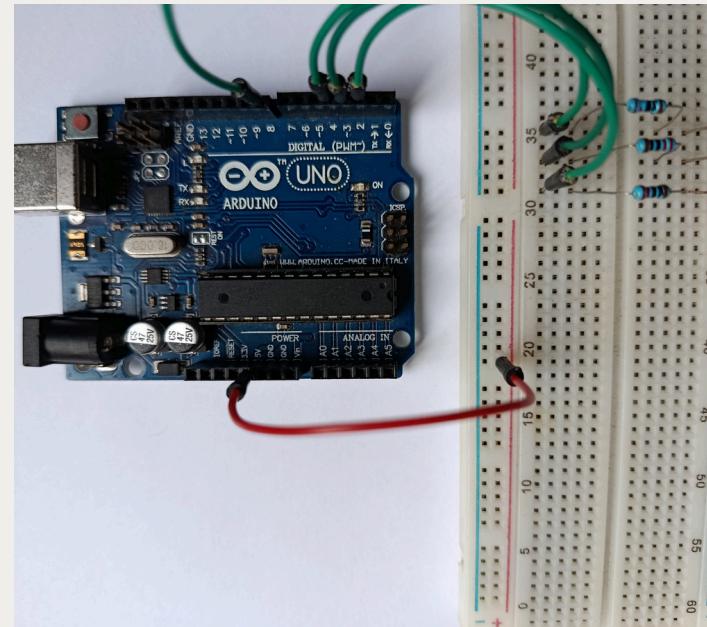
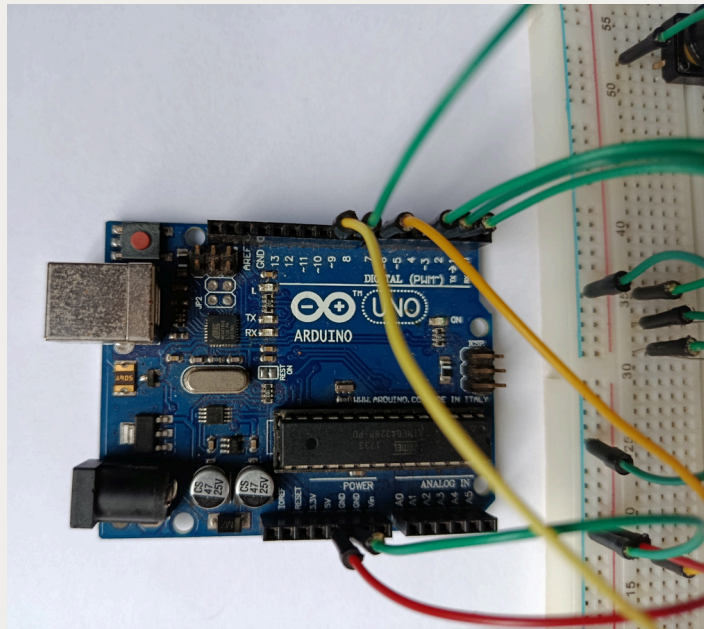


Diagrama de Conexiones y Cableado

Alimentación Principal (Voltaje y Tierra):

- Línea Positiva (5V): Cable rojo desde la placa Arduino. Pasa corriente al otro lado de la protoboard con un puente de cable verde para alimentar las líneas verticales.
- Línea Negativa (GND): Cable blanco usado como puente hacia la línea negativa de abajo para cerrar el circuito de los componentes.
- Bloque de los LEDs e Interruptor:
- LEDs: Tienen sus patitas negativas conectadas a la línea común de tierra mediante el puente blanco.
- Botón: Conectado a su pin digital con un cable verde (junto a sus resistencias) para mandar la señal de activación.
- Bloque de los Servomotores (Efecto Espejo):
- Señal Lógica: Cables amarillos que van desde los pines digitales (7 y 9) hacia los motores para decirles a qué posición moverse.
- Alimentación de los Servos: Cable chocolate (marrón) o blanco para el negativo, y cable rojo para el positivo de 5V.
- Bloque del Joystick (Control Analógico):
- Alimentación: Cable rojo para el negativo y cable blanco para el positivo (invertidos por los puertos de entrada del componente).
- Señal Analógica: Cable naranja con combinación amarilla (reparado) conectado directamente al pin analógico A0.



04 / ESTRUCTURA

Declaración de Variables y Biblioteca

- `#include <Servo.h>`: Usamos los corchetes menor y mayor (< >). Es la librería que nos permite usar y controlar los servos.
- `Servo miServo1;` y `Servo miServo2;`: Definimos dos motores separados para que no se confundan en la lógica, ya que uno va a ser el "espejo" del otro.
- `bool` (Botón): Una función que guarda valores como verdadero o falso. Nos sirve para que al pulsar el botón en un minisegundo (300 ms), se guarde y cambie esa función.
- `const int` (Pin 8): Hace que el valor del pin no cambie aunque use la variable en otro lado. Si la repito o intento cambiarla, el código manda un error.
- Variables `int`: Creamos una en valor cero para el pin analógico A0 (joystick) y otra variable `ángulo = 0` para empezar desde la nada.

Configuración en el void `setup()`

- (Esta parte arranca una sola vez al iniciar)
- `INPUT_PULLUP`: Configura el botón con una resistencia incorporada que ya trae el Arduino.
- Pines 2, 3 y 4: Los declaramos como `OUTPUT` porque son las salidas para los tres LEDs.
- `.attach(7)` y `.attach(9)`: Le declaramos a `miServo1` y `miServo2` en qué pines físicos están conectados.
- `Serial.begin(9600);`: Lo iniciamos para poder escribir y ver los datos en la pantalla.

```
1  #include <Servo.h>
2
3  Servo miServo1;
4  Servo miServo2;
5  bool boton = false;
6  const int PIN_BOTON = 8;
7  int valor = 0;
8  int pin = A0;
9  int angulo = 0;
10
11 void setup() {
12     pinMode(PIN_BOTON, INPUT_PULLUP);
13     pinMode(2, OUTPUT);
14     pinMode(3, OUTPUT);
15     pinMode(4, OUTPUT);
16     miServo1.attach(7);
17     miServo2.attach(9);
18     Serial.begin(9600);
19 }
20
```



ESTRUCTURA

Lógica del Interruptor en el void loop()

- `int estadoBoton = digitalRead(PIN_BOTON);`: El código empieza leyendo el estado físico del pin del botón para saber si alguien lo está tocando.
- `if (estadoBoton == LOW)`: Como configuramos el pin con resistencia interna, cuando presionamos el botón el Arduino detecta un voltaje bajo (LOW) y se activa esta condición.
- El sistema de "Switch" (if / else):
 - Si la variable `boton` está en falso (`false`), el código la cambia a verdadero (`true`).
 - Si no (`else`), significa que ya estaba en verdadero, así que la regresa a falso (`false`).
- `delay(250);`: Le damos una pausa de 250 milisegundos justo después de presionar. Esto sirve para que el Arduino guarde el cambio con calma y no lea la misma pulsación un montón de veces por error.

```
21 void loop() {
22     int estadoBoton = digitalRead(PIN_BOTON);
23
24     if (estadoBoton == LOW) {
25         if (boton == false) {
26             boton = true;
27         }
28         else {
29             boton = false;
30         }
31         delay(250);
32     }
```



ESTRUCTURA

Lógica de Control: Servos y LEDs

- Condición Principal (if (boton == true)): Si el botón está en verdadero, se desbloquea el mecanismo y empieza a revisar el pin del joystick.
- Lectura y Mapeo (map): Guarda lo que lee el joystick en la variable valor y lo convierte (con la función map) de su rango analógico (0 a 1023) a ángulos para los motores (0 a 180°).
- Movimiento Espejo de los Servos:
 - miServo1.write(angulo); -> Se mueve al ángulo que da el joystick.
 - miServo2.write(180 - angulo); -> Hace la posición contraria (el espejo).
- Monitoreo Serie y LEDs de Alerta:
 - Muestra en pantalla el mensaje "activar / ángulo:" junto con los grados en tiempo real.
 - Apaga el LED amarillo (Pin 3) y el LED amarillo (Pin 4) al activarse.
 -
- Condición de Ángulo (Menor o Mayor a 90°):
 - Si es menor o igual a 90°: Apaga la luz verde y enciende la roja.
 - Si es mayor a 90°: Apaga la luz roja y enciende la verde por unos segundos.
 -
- Mecanismo Bloqueado (else): Si la condición del botón es falsa, muestra en pantalla "desactivado", apaga los LEDs 2 y 4, deja encendido el amarillo (Pin 3), regresa ambos servos a 0° y hace una espera de 300 ms.

```
35  if (boton == true) {
36      valor = analogRead(pin);
37      angulo = map(valor, 0, 1023, 0, 180);
38      miServo1.write(angulo);
39      miServo2.write(180 - angulo);
40      |
41      Serial.print("[Activo] | Angulo : ");
42      Serial.println(angulo);
43      digitalWrite(3, LOW);
44
45      if (angulo <= 90) {
46          digitalWrite(4, LOW);
47          digitalWrite(2, HIGH);
48      }
49      else {
50          digitalWrite(2, LOW);
51          digitalWrite(4, HIGH);
52      }
53      delay(15);
54  }
55  else {
56      Serial.println("[Desativado]");
57      digitalWrite(2, LOW);
58      digitalWrite(4, LOW);
59      digitalWrite(3, HIGH);
60      miServo1.write(0);
61      miServo2.write(0);
62      delay(300);
63  }
64 }
```



GRACIAS
POR VER

